

Deploying a Flask-React App to Render

 [_ \(https://github.com/learn-co-curriculum/python-p4-deploying-flask-react-app-to-render\)](https://github.com/learn-co-curriculum/python-p4-deploying-flask-react-app-to-render)  [_ \(https://github.com/learn-co-curriculum/python-p4-deploying-flask-react-app-to-render/issues/new\)](https://github.com/learn-co-curriculum/python-p4-deploying-flask-react-app-to-render/issues/new)

Learning Goals

- Understand the React build process and how to serve a React app from Flask.
- Understand challenges of client-side routing in a deployed application.
- Deploy a full-stack Flask-React application to Render.

Key Vocab

- **Deployment:** the processes that make an application available for its intended use. For web applications, this means moving the application to a platform that supports requests from the internet.
- **Developer Operations (DevOps):** the practices and tools that improve a team's ability to develop and deploy applications quickly.
- **PostgreSQL:** an open-source relational database system that provides more SQL functionality than SQLite. Unlike SQLite, its data is stored on a server rather than in files.
- **Platform as a Service (PaaS):** a development and deployment platform that exists on a wide range of servers with different functionality. PaaS solutions reduce maintenance time for a software development team, but can increase cost. Some PaaS solutions, such as Render, provide free tiers for small applications.

Introduction

In the previous lesson, we deployed a small Flask API application to Render to learn how the deployment process works in general, and what steps are required to take the code from our machine and get it to run on a server.

In this lesson, we'll be tackling a more complex app with a modern Flask-React stack, and explore some of the challenges of getting these two apps to run together on a single server.

Setup

To follow along with this lesson, we have a pre-built Flask-React application that you'll be deploying to Render. To start, [fork this repo from GitHub](https://github.com/learn-co-curriculum/python-p4-deployment-bird-app/fork)  [_\(https://github.com/learn-co-curriculum/python-p4-deployment-bird-app/fork\)_](https://github.com/learn-co-curriculum/python-p4-deployment-bird-app/fork).

After downloading the code, set up the repository locally:

```
$ npm install --prefix client
$ pipenv install && pipenv shell
```

Create a `.env` file at root and add the following variable:

```
DATABASE_URI=postgresql://{retrieve this from from render}
```

We've installed a new package in this repository called `python-dotenv`. It allows us to set environment variables when we run our application using `.env` files. This is a nice midway point between setting invisible environment variables from the command line and writing hidden values into our code.

To generate these environment variables, we just need to run the following command at the beginning of the module:

```
# server/app.py

from dotenv import load_dotenv
load_dotenv()
```

After this, we can import any of our `.env` variables with `os.environ.get()`.

Run the following commands to install upgrade and seed our database:

```
$ cd server  
$ flask db upgrade  
$ python seed.py
```

This application has a RESTful Flask API, a React frontend using React Router for client-side routing, and PostgreSQL for the database.

You can now run the app locally with:

```
$ honcho start -f Procfile.dev
```

Spend some time familiarizing yourself with the code for the demo app before proceeding. We'll be walking through its setup and why certain choices were made through the course of this lesson.

React Production Build

One of the great features that Create React App provides to developers is the ability to build different versions of a React application for different environments.

When working in the **development** environment, a typical workflow for adding new features to a React application is something like this:

- Run `npm start` to run a development server.
- Make changes to the app by editing the files.
- View those changes in the browser.

To enable this *excellent* developer experience, Create React App uses [webpack](https://webpack.js.org/)  (<https://webpack.js.org/>) under the hood to create a development server with hot module reloading, so any changes to the files in our application will be instantly visible to us in the browser. It also has a lot of other nice features in development mode, like showing us good error and warning messages via the console.

Create React App is *also* capable of building an entirely different version of our application for **production**, also thanks to webpack. The end goal of our application is to get it into the hands of our users via our website. For our app to run in production, we have a different set of needs:

- **Build** the static HTML, JavaScript and CSS files needed to run our app in the browser, keeping them as small as possible.

- **Serve** the application's files from a server hosted online, rather than a local webpack development server.
- Don't show any error messages/warnings that are meant for developers rather than our website's users.

Building a Static React App

When developing the frontend of a site using Create React App, our ultimate goal is to create a **static site** consisting of pre-built HTML, JavaScript, and CSS files, which can be served by Flask when a user makes a request to the server to view our frontend. To demonstrate this process of **building** the production version of our React app and **serving** it from the Flask app, follow these steps.

1. Build the production version of our React app:

```
$ npm run build --prefix client
```

This command will generate a bundled and minified version of our React app in the `client/build` folder.

Check out the files in that directory, and in particular the JavaScript files. You'll notice they have very little resemblance to the files in your `src` directory! This is because of that **bundling** and **minification** process: taking the source code you wrote, along with any external JavaScript libraries your code depends on, and squishing it as small as possible.

2. Add static routes to Flask:

If you check `app.py`, you will see that the following additions have been made since you last saw the bird API:

```
app = Flask(  
    __name__,  
    static_url_path='',  
    static_folder='../client/build',  
    template_folder='../client/build'  
)
```

...

```
@app.errorhandler(404)  
def not_found(e):
```

```
return render_template("index.html")
```

These configure our Flask app for where to search for static and template files- both in our `client/build/` directory.

We also set up a catch-all here for any route that doesn't match those already defined on the server. This means when Flask receives a request, it will render the `index.html` that was generated to run the client application. The client still handles its own routing through clicks and form submissions, but with this configuration, Flask can find the resources by URL as well. This is important for when people refresh the page, or visit your site, either manually, from bookmarks, or an external link.

NOTE: Often, you may be setting up RESTful client-side routes, allowing people to go to `/birds` or `/birds/:id` to see all of the birds, or one at a time, respectively. These routes wouldn't be accessible on the frontend if they're already set up on the server (like they are in this app). To solve this, it's common to rewrite the backend routes so they all start with `/api/`, like `/api/birds` and `/api/birds/<int:id>` in order to free up the non-api urls to be used for client side routing. Just remember to also update your fetches to match backend urls.

3. Run the Flask server:

```
$ gunicorn --chdir server app:app
```

Visit <http://localhost:8000>  <http://localhost:8000> in the browser. You should see the production version of the React application!

Explore the React app in the browser using the React dev tools. What differences do you see between this version of the app and what you're used to when running in development mode?

Now you've seen how to build a production version of the React application locally, and some of the differences between this version and the development version you're more familiar with.

Now that you've seen how to create a production version of our React app locally and integrated it with Flask, let's talk about how to deploy the application to Render.

Render Build Process

Think about the steps to build our React application locally. What did we have to do to build the React application in such a way that it could be served by our Flask application? Well, we had to:

- Run `npm install --prefix client` to install any dependencies.
- Use `npm run build --prefix client` to create the production app.
- Install `pip` dependencies for the Flask app.
- Run `gunicorn --chdir server app:app` to run the Flask server.

We would also need to repeat these steps any time we made any changes to the React code, i.e., to anything in the `client` folder. Ideally, we'd like to be able to **automate** those steps when we deploy this app to Render, so we can just push up new versions of our code to Render and deploy them like we were able to do in the previous lesson.

Thankfully, Render lets us do just that! Let's get started with the deploying process and talk through how this automation works.

Commit all of your work to your fork on GitHub and copy the project URL.

Navigate to [your Render dashboard][<https://dashboard.render.com>  (<https://dashboard.render.com>)] and create a new web service. Find your forked repository through "Connect a repository" or search by the copied URL under "Public Git repository."

Change your "Build Command" to the following:

```
$ pip install -r requirements.txt && npm install --prefix client && npm run build --prefix client
```

Change your "Start Command" to the following:

```
$ gunicorn --chdir server app:app
```

These commands will:

- Install any Python dependencies with `pip`.
- Install any Node dependencies with `npm`.

- Build your static site with `npm`.
- Run your Flask server.

Once you have saved these changes, navigate to the "Environment" tab and make sure the following values are set:

```
DATABASE_URI=postgresql://{retrieve this from from render}  
PYTHON_VERSION=3.8.13
```

Click "Save Changes" and wait for a while. (Render's free tier can take up to an hour to get everything set up, so you might want to grab a bagel and coffee while you wait.) Navigate to "Events" to check for progress and errors. When Render tells you the site is "Live", navigate to your site URL and view Birdsy in all its glory!

Conclusion

Creating a website out of multiple applications working together adds a significant amount of complexity when it comes time to deploy our application. The upside to this approach is we get to leverage the strengths of each of the tools we're using: React for a speedy, responsive user interface, and Flask for a robust, well-designed backend to communicate with the database.

By spending some time upfront to understand and automate parts of the deployment process, we can make future deployments simpler.

For your code challenges using a React frontend and Flask API backend, we'll provide a template project to use so you don't have to worry about configuring the tricky parts of the deployment process yourself. However, it's helpful to have an understanding of this configuration should you wish to customize it or troubleshoot issues related to deployments in the future.

Resources

- [Deploy a Flask App - Render](https://render.com/docs/deploy-flask)  (https://render.com/docs/deploy-flask)
- [Deployment - Create React App](https://create-react-app.dev/docs/deployment/#:%7E:text=npm%20run%20build%20creates%20a,.js%20file.)  (https://create-react-app.dev/docs/deployment/#:%7E:text=npm%20run%20build%20creates%20a,.js%20file.)
- [JavaScript, `fetch`, and JSON - Pallets](https://flask.palletsprojects.com/en/2.2.x/patterns/javascript/)  (https://flask.palletsprojects.com/en/2.2.x/patterns/javascript/)